

FORMATION EMBARQUÉE

Module 05 — Java

Formation Systèmes embarqués & programmation bas niveau

Systèmes embarqués · bas niveau · automatismes

Module 05 — Java : la POO complète et son rôle autour de l'embarqué

Java ne tourne pas sur un microcontrôleur 8 bits — il lui faut une machine virtuelle (JVM). Son rôle dans ton parcours : **au-dessus** de l'embarqué — supervision industrielle, applications Android qui pilotent tes objets, serveurs qui reçoivent les données IoT, outillage. Et c'est un excellent professeur de POO, qui consolidera ton C++.

Installation : un JDK (Temurin/OpenJDK 21), et IntelliJ IDEA Community ou VS Code. Compiler/exécuter à la main :

```
javac Main.java # → Main.class (bytecode)
java Main       # exécuté par la JVM
```

 **Mini-TP associé** (15-30 min, sans matériel) : [mini-tp/java/](#) — un décodeur de trame et le piège du byte signé. À faire **après la première lecture**, avant les exercices.

1. Java vs C/C++ : ce qui change

	C/C++	Java
Compilation	code machine natif	bytecode exécuté par la JVM (portable)
Mémoire	manuelle (<code>malloc</code> / <code>new</code> + <code>delete</code>)	ramasse-miettes (garbage collector)
Pointeurs	oui, arithmétique incluse	non — références seulement, pas d'accès mémoire brut
Héritage multiple	oui (C++)	non (mais interfaces multiples)
Débordements de tampon	ton problème	exception <code>ArrayIndexOutOfBoundsException</code>
Où ça tourne	partout, y compris nu	là où il y a une JVM (PC, serveur, Android)

Conséquence : impossible d'écrire un driver en Java, mais tu ne chasseras jamais un pointeur fou.

2. Bases du langage

```
public class Main {
    public static void main(String[] args) { // point d'entrée
        int n = 42; // types primitifs : int(32b), long(64b),
        double d = 3.14; // short, byte, float, double, boolean, char
        boolean ok = true;
        String nom = "capteur"; // String = objet immuable

        // Attention embarqué : byte est SIGNÉ (-128..127), pas de unsigned en Java !
        int nonSigné = octet & 0xFF; // le masque classique pour "désigner" un octet

        if (n > 10) System.out.println("grand");
        for (int i = 0; i < 5; i++) { }
        while (ok) { break; }
        switch (n) { case 42 -> System.out.println("réponse"); default -> {} }

        int[] tab = {1, 2, 3};
        for (int v : tab) System.out.println(v); // for-each
    }
}
```

3. POO à la Java

3.1 Classes, encapsulation

```
public class Capteur {
    private final String nom; // final = affecté une fois (comme const)
    private double derniereMesure;

    public Capteur(String nom) { // constructeur
        this.nom = nom;
    }

    public double lire() {
        derniereMesure = acquerir();
        return derniereMesure;
    }

    public String getNom() { return nom; } // accesseur (convention JavaBeans)

    private double acquerir() { return Math.random() * 100; }
}
```

Une classe publique par fichier, fichier du même nom (`Capteur.java`). Tout objet se crée avec `new` et vit sur le tas — le GC le libère quand plus personne ne le référence.

3.2 Héritage, interfaces, polymorphisme

```
public interface Mesurable {                                // interface = contrat pur
    double lire();
    default String unite() { return "u"; } // méthode par défaut possible
}

public class CapteurTemperature extends Capteur implements Mesurable {
    public CapteurTemperature() { super("temp"); }

    @Override
    public double lire() { return super.lire() / 4.0; }

    @Override
    public String unite() { return "°C"; }
}

List<Mesurable> capteurs = List.of(new CapteurTemperature(), new CapteurPression());
for (Mesurable c : capteurs)
    System.out.println(c.lire() + " " + c.unite()); // polymorphisme
```

En Java **toutes les méthodes sont virtuelles** par défaut (contrairement à C++). Les interfaces remplacent l'héritage multiple : une classe en implémente autant qu'elle veut. Autres briques : classes `abstract`, `record` (classe de données immuable en une ligne : `record Mesure(String capteur, double valeur, long ts) {}`), `enum` riches (avec champs et méthodes).

4. Exceptions et robustesse

```
try {
    int v = Integer.parseInt(texte);
} catch (NumberFormatException e) {
    System.err.println("Entrée invalide : " + e.getMessage());
} finally {
    // toujours exécuté (nettoyage)
}

// try-with-resources = le RAII de Java (ferme automatiquement)
try (var port = new Socket("192.168.1.50", 502)) {
    // ... parler Modbus TCP à un automate ...
} // socket fermé ici, même en cas d'exception
```

Les exceptions **vérifiées** (`IOException` ...) doivent être attrapées ou déclarées (`throws IOException`) — le compilateur y veille.

5. Collections et génériques

```
List<Double> mesures = new ArrayList<>(); // tableau dynamique
mesures.add(21.5);

Map<String, Capteur> parNom = new HashMap<>(); // dictionnaire
parNom.put("temp", capteurTemp);

Deque<Byte> fifo = new ArrayDeque<>(); // file (ton ring buffer, en mieux)

// Streams : traiter des données de capteurs en une passe déclarative
double moyenne = mesures.stream()
    .filter(v -> v > 0)
    .mapToDouble(Double::doubleValue)
    .average()
    .orElse(0.0);
```

Les génériques (`List<Double>`) = les templates de C++, en plus simple (mais effacés à l'exécution).

6. Threads : la concurrence sans registre

```
Thread lecteur = new Thread(() -> {
    while (!Thread.currentThread().isInterrupted()) {
        double v = capteur.lire();
        file.offer(v); // file thread-safe
        try { Thread.sleep(1000); } catch (InterruptedException e) { return; }
    }
});
lecteur.start();
```

Outils clés : `synchronized` (section critique), `AtomicInteger`, `BlockingQueue` (producteur/consommateur — le pattern des passerelles IoT), `ExecutorService` (pool de threads). Les concepts (course critique, section critique, atomicité) sont **exactement ceux des ISR** du module 00 — Java te les fait pratiquer confortablement.

7. Java et le monde embarqué/industriel : cas concrets

7.1 Parler à un automate en Modbus TCP

Avec une bibliothèque comme *digitalpetri/modbus* ou *j2mod* :

```
// Pseudo-code représentatif (API j2mod simplifiée)
ModbusTCPMaster maitre = new ModbusTCPMaster("192.168.1.50");
maitre.connect();
Register[] regs = maitre.readMultipleRegisters(0, 10); // 10 mots à l'adresse 0
int temperature = regs[0].getValue();
maitre.writeSingleRegister(100, new SimpleRegister(1)); // commander une sortie
```

C'est le pont typique entre tes modules 07/08 (automates) et un logiciel de supervision maison.

7.2 Recevoir des mesures IoT en MQTT (Eclipse Paho)

```
MqttClient client = new MqttClient("tcp://broker:1883", "superviseur");
client.connect();
client.subscribe("usine/four1/temperature", (topic, msg) -> {
    double t = Double.parseDouble(new String(msg.getPayload()));
    if (t > 80) alerter(t);
});
```

L'ESP32 du module 03 publie ; ton serveur Java s'abonne, historise (JDBC → PostgreSQL), alerte, affiche.

7.3 Où Java apparaît dans l'industrie

- **Android** : applis qui pilotent tes cartes en Bluetooth/Wi-Fi (Kotlin, cousin direct de Java).
- **Supervision/SCADA** : Ignition (plateforme SCADA très répandue) est en Java ; ses scripts sont en Jython.
- **Serveurs & passerelles** : Spring Boot pour exposer les données machines en API REST ; Eclipse Kura pour les passerelles IoT.
- **Outils** : beaucoup d'outils EDA/IDE (dont Eclipse, la base de STM32CubeIDE et de TIA-like) sont écrits en Java.

8. Mini-projet : superviseur de station météo

Complète le projet fil rouge du module 03 :

1. L'ESP32 publie `{"t":21.5,"h":48}` sur le topic MQTT `meteo/salon`.
2. Application Java : s'abonne, déséréalise le JSON (bibliothèque Jackson), stocke en base SQLite (JDBC), calcule min/max/moyenne glissante.
3. Expose `GET /mesures/dernieres` en HTTP (un simple `com.sun.net.httpserver.HttpServer` suffit pour commencer).
4. Bonus : seuil d'alerte configurable + envoi d'un e-mail.

Tu auras construit une **chaîne IoT complète** : capteur → firmware C++ → réseau → back-end Java.

Exercices

1. Modélise `Capteur` (abstraite), `CapteurTemp`, `CapteurHum`, et une classe `Station` qui interroge une `List<Capteur>` et journalise en CSV.
2. Implémente producteur/consommateur : un thread « acquisition » (simulé), un thread « traitement » avec `BlockingQueue`, arrêt propre par interruption.
3. Écris un décodeur de trame : `byte[] {0xA5, id, hi, lo}` → objet `Mesure` (attention au signe des `byte` : masque `& 0xFF` !).
4. Parseur de log : lis un fichier CSV de mesures, sors min/max/moyenne par capteur avec les streams.

→ Suite : [Module 06 — Autres langages utiles.](#)